Lab 3: Component Modelling & Architectural Pattern Selection

Architecture Justification - Self-Service Coffee Kiosk System

Architecture Selection: We chose **Layered Architecture** for the **Self-Service Coffee Kiosk** System.

1. Architectural Choice

The kiosk is built in three horizontal layers. The **Presentation Layer** holds the User Interface Component, which handles touch-screen input. The **Business Layer** holds the Order Manager, Payment Service and Receipt Printer components. The **Data Layer** holds the Database Component, which stores menu and pricing data. Components talk only through defined interfaces (IOrder, IPayment, IReceipt, IDatabase), and the Order Manager coordinates every request.

2. Reasons for the Choice

Reason 1 - The kiosk's requirements map directly onto layers. The touch-screen interface, the order and pricing logic (3 coffee types x 2 sizes), credit-card payment, receipt printing and menu storage each belong to exactly one component. Because each component hides its internals behind an interface, parts can be replaced independently. For example, a different card reader or printer model can be swapped in without changing the UI or the order logic, and a new drink only touches the data and business layers.

Reason 2 - The system is small, single-device and fixed in scope. One kiosk with a simple ordering flow does not need independently scaled services. Microservices would add network calls, deployment overhead and data-consistency problems for no benefit. A plain Client-Server design would add a central server as a single point of failure, even though the kiosk must keep selling coffee on its own. A layered design is simple to build, test and deploy on one machine, and easy for a small team to understand.

3. Security Advantage

Layering enforces controlled access. The User Interface has no direct access to the Payment Service or the database. Every request must pass through the Order Manager via the IOrder interface, so input can be validated in one place before it reaches sensitive components. Card details are handled only inside the Payment Service, which communicates with the payment provider. They are never stored in the menu database, and the receipt shows no full card number. This confines the sensitive data to one component and makes it easier to secure and audit.

4. Performance Benefit

All layers run on the same kiosk, so calls between components are local, in-process calls with no network hop. This keeps touch-screen response fast. The menu and pricing data is small, so the Order Manager can load it from the database once at start-up and serve later selections from memory, avoiding repeated queries during an order. Payment is the only step that waits on an external service, and the layered design keeps that wait isolated inside the Payment Service so the rest of the interface stays responsive.

Trade-off acknowledged: layered architectures can add some call overhead and make individual components harder to scale. At the scale of a single kiosk this cost is negligible compared with the gains in simplicity, security and maintainability.